GameHub Architecture Specification v3

High-Scale Production System Blueprint

Principal Software Architect, Staff Engineer, DevOps Architect, Security Architect

June 21, 2026

Contents

Chapter 1

Executive Summary & System Typologies

1.1 Introduction & System Goals

The GameHub system is transitioning from a Minimum Viable Product (MVP) to a high-scale production deployment. The system serves as a unified campus sports, matchmaking, and reservation portal. Key system constraints include strict consistency for ELO ledgers, low-latency matching, physical resource constraints (play area bookings), offline support for mobile PWA platforms, and high-concurrency event scoring.

1.1.1 Physical Resource Sharing & Limits (Admin-Configured)

- **Dynamic Physical PlayAreas**: The total count and distribution of physical assets (Pebolim, PingPong, Snooker, Bola 8, etc.) are dynamically managed by administrators rather than being hardcoded in application logic, schemas, or initial seeds. Administrators can dynamically create, assign, and alter physical table instances.
- **Multi-Game Shared Resource Mapping**: A single physical play area (e.g., a billiard table) can support multiple game types simultaneously (e.g., both `BOLA_8` and `SNOOKER`).
- **Conflict Blocking Policy**: Reserving or booking a timeslot for a specific game type on a shared physical play area dynamically blocks all other supported game types on that specific play area for that exact time window to prevent overlapping reservation conflicts.

1.1.2 Virtual Space Lifecycle

- **Card Games (TRUCO, BURACO)**: Card games require zero physical infrastructure. They support programmatic dynamic initialization and immediate automatic deletion upon match finalization. These transient virtual spaces completely bypass physical slot allocation validations, database checks, and optimistic concurrency control (OCC) constraints.

1.2 Data Privacy Anonymization & Identity Scrubbing Strategy

To comply with global data protection regulations while preserving historical competitive data, the Game Hub defines a strict strategy boundary for user account deletions:

- **Identity Scrubbing**: All sensitive personally identifiable information (PII) including `fullName`, `email`, and `nusp` is permanently wiped or replaced with randomized anonymized placeholders (e.g., `deleted_user_12345`).

- **Relational Integrity**: The historical records in the `EloLedger`, `MatchParticipant` score history, team roster logs, and administrative audit trails remain intact. Relational foreign keys pointing to the user's UUID are preserved to prevent data corruption and ensure seasons stats and ELO progression indices of other players are not altered.

1.3 Unified Business Capabilities Map

The business design is driven by six core capabilities that represent the functional scope of the Game Hub:

1. **Identity Management (RBAC)**: Governs registration, Course and Institute boundaries, User preferences, notification registration, and Role-Based Access Control (RBAC) claims.
2. **Facility & Equipment Coordination**: Coordinates physical recreation areas (PlayAreas), schedules reservations, controls hardware loans (Equipments, such as billiard cues and ping-pong paddles), and manages maintenance statuses.
3. **Core Matchmaking & Match Engine**: Manages active queuing systems, validates player ratings, tracks live heartbeats, records match scores, and coordinates set progressions.
4. **Competition Slots & Brackets**: Coordinates tournament match slots (and brackets) and calculates progression states for academic tournaments (elimination and round-robin).
5. **ELO Engine Progression**: Evaluates match outcome details to execute rating adjustments at the close of active game sets, tracking ranking progression across competitive seasons.
6. **Moderation & Social Telemetry**: Governs friendships, peer communication channels, dispute reviews for contested match outcomes, and automatic user sanctions for negative behaviors.

1.4 Modular Monolith Structural Design

To maintain developer velocity, minimize operations overhead, and prevent premature microservices decomposition, GameHub is designed as a structured Modular Monolith using NestJS. The monolith is composed of independent, loosely coupled modules with strict boundaries. Communication between modules is mediated through well-defined module APIs, asynchronous in-memory event buses, or strict transaction-boundary contracts.

1.4.1 Domain Boundaries

The domain boundaries are aligned with Domain-Driven Design (DDD) principles:

1. **Identity & Teams Domain**: Manages users, roles, team rosters, and membership state.
2. **Facilities Domain**: Governs courts, tables, play areas, and timeslot-based reservations.
3. **Matchmaking Domain**: Handles active matching tickets, grouping algorithms, queue management, and pairing triggers.
4. **Matches & Progression Domain**: Manages match lifecycles, ELO updates via transactional ledger entries, and seasonal rankings.
5. **Events & Tournaments Domain**: Governs single/double elimination bracket structures, event configurations, and score summaries.
6. **Social & Moderation Domain**: Manages dispute resolutions, real-time match chats, report logs, and user access sanctions.

1.4.2 Module Boundary Enforcement Rules

To prevent the codebase from degrading into a "big ball of mud", the following architectural constraints are enforced:

- **Circular Dependencies**: Prohibited. Checked at build time via dependency-cruiser and linting rules.
- **Database Access**: Each module owns its schema tables. Cross-module database queries or joins are strictly forbidden. Access to data owned by another module must go through that module's exported service API.
- **Shared Kernels**: Limited to shared types, utility helpers, and common domain events.

1.5 Hexagonal Architecture (Ports & Adapters)

Within each module, dependency inversion is maintained using Hexagonal Architecture. The core business logic is isolated from external frameworks, database drivers, and communication protocols.

1.5.1 Structural Layers

1. **Domain Model (Core)**: Contains pure business logic, entities, value objects, and domain exceptions. No external dependencies.
2. **Ports (Interfaces)**:
    - *Inbound Ports (Use Cases)*: Define commands, queries, and orchestration flows.

- *Outbound Ports (Gateways)*: Define abstract interfaces for persistence, message queues, push notification services, and cache operations.

3. **Adapters (Infrastructure)**:
   - *Inbound Adapters (Primary)*: REST controllers, Socket.io gateways, background job consumers.
   - *Outbound Adapters (Secondary)*: PostgreSQL repositories via TypeORM, Redis caching operations, BullMQ job triggers.

1.6 Core Data Flows

1.6.1 Matchmaking to Match Creation Flow

1. A `User` submits a `MatchmakingTicket`.
2. The Matchmaking module places the ticket in a Redis-backed queue managed by BullMQ.
3. A matchmaking worker aggregates tickets and groups users based on matching criteria (skill rating, location, latency).
4. On pairing, a `Match` entity is initialized in a `PENDING` state.
5. The Matchmaking module invokes the Facilities module to provision a `PlayAreaReservation` on a matching `PlayArea`.
6. Once reservation is confirmed, the `Match` moves to `SCHEDULED` state.
7. Players are notified via real-time WebSocket connection.

1.6.2 Match Finalization, Progression, and Score Flow

1. The match is completed. Match scores are submitted.
2. The Match module marks the match as `COMPLETED`.
3. The Match module publishes a `MatchFinalizedEvent`.
4. **Progression Module Listener**:
   - Appends ELO change vectors into the transactional `EloLedger`.
   - Recalculates `PlayerRanking` scores.
5. **Events Module Listener**:
   - Updates tournament bracket trees if the match is part of an active `Tournament`.
   - Computes points and updates `EventScore` records.

1.6.3 Dispute and Sanction Workflow

1. If users report incorrect results, a `MatchDispute` is created.
2. Creating a dispute transitions the associated ELO updates in the ELO ledger to a `LOCKED` state.
3. Moderation staff are notified via admin dashboards.

4. If malicious behavior is confirmed, a `UserSanction` is placed against the offending user, blocking matchmaking access.

5. Once resolved, ELO ledger records are unlocked and corrected.

1.7 State Machine & Lifecycle Resilience

Mobile systems frequently experience network drops and thread suspensions during backgrounding transitions. To prevent false-positive forfeits, the NestJS gateway processes the client-side `minimize_presence` lifecycle event:

- **Event Capture**: The mobile web client (wrapped via Capacitor) listens to the OS lifecycle changes. When backgrounding is detected, it broadcasts `minimize_presence` to the WebSocket gateway.
- **Dynamic TTL Extension**: The gateway catches this event and extends the associated player's Redis heartbeat TTL key from the default 10 seconds to 60 seconds.
- **Worker Actions**: If the player resumes within 60 seconds, a standard heartbeat resets the TTL to 10 seconds. Otherwise, the status is flagged as disconnected, triggering the forfeit flow.

1.8 Storage & Concurrency Controls

To avoid booking overlaps under high concurrency (e.g., peak tournament hours), the system implements the "Matched but Homeless" recovery strategy using Optimistic Concurrency Control (OCC) and transactional retries.

1.8.1 TypeORM Mapping Template with Versioning

The entity model for `PlayArea` employs an optimistic version control structure:

```
import { Entity, PrimaryGeneratedColumn, Column, VersionColumn, OneToMany }
    from "typeorm";
import { PlayAreaReservation } from "./PlayAreaReservation.entity";

@Entity("play_areas")
export class PlayArea {
  @PrimaryGeneratedColumn("uuid")
  id: string;

  @Column({ type: "varchar", length: 100 })
  name: string;

  @Column({ type: "boolean", default: true })
  isActive: boolean;

  @VersionColumn()
  version: number;

  @OneToMany(() => PlayAreaReservation, (reservation) => reservation.playArea)
  reservations: PlayAreaReservation[];
}
```

Listing 1.1: PlayArea Entity TypeORM Definition

1.8.2 OCC Fallback & Recovery Control Flow

1. A matchmaking match is paired and scheduled.
2. In a transaction context, the allocation worker attempts to insert a `PlayAreaReservation` linking the `PlayArea` at timeslot T.
3. The worker checks the version and updates the `PlayArea` using: `UPDATE play_areas SET version = version + 1 WHERE id = :id AND version = :version`
4. If the update count returns 0, the system throws an `OptimisticLockFailedException`.
5. **Recovery Routine**: The match creation transaction rolls back immediately. The players' tickets are pushed back to the front of the active BullMQ queue using a high-priority job configuration, ensuring immediate rematching.

1.9 Security & Authentication Architecture

The system enforces a strict dual-token authentication model, incorporating native client storage fallback paths to handle WebKit Intelligent Tracking Prevention (ITP) restrictions on mobile iOS shells.

1.9.1 Authentication Flow & PIN Validation

- **4-Digit Numerical PIN Validation**: The system profile authentication utilizes a 4-digit numerical PIN (securely hashed, e.g., via Argon2) rather than a generic alphanumeric password string, facilitating rapid and secure entry on physical terminal kiosks and mobile shells.
- **Web Client Mode**: On successful login via valid 4-digit PIN verification, the server returns a short-lived access token in the JSON body, and sets a long-lived refresh token in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie.
- **Mobile Shell (Capacitor) Mode**: WebKit ITP frequently purges or blocks third-party and local cookies on hybrid mobile wrappers. To mitigate this, the API auth routing detects the user agent header. If a mobile shell is identified, it bypasses the HTTP cookie setup and returns both the access and refresh tokens inside the encrypted JSON response body.
- **Native Secure Storage**: The Capacitor shell captures these tokens and saves them using the native `CapacitorSecureStorage` adapter, which writes directly to the iOS Keychain or Android KeyStore. Subsequent requests inject the refresh token inside the HTTP `Authorization` headers during session updates.

1.10 Notification Decoupling

The application core contains no dependency on external notification endpoints (FCM, APNs). It interacts solely via ports.

1.10.1 INotificationServicePort Interface

```
export interface INotificationServicePort {
  sendPushNotification(
    targetToken: string,
    title: string,
```

```
    body: string,
    metadata?: Record<string, string>
  ): Promise<boolean>;
}
```

Listing 1.2: Push Notification Port

1.10.2 FCM Adapter Contract Payload

```
{
  "message": {
    "token": "fcm_token_xyz123",
    "notification": {
      "title": "Match Scheduled",
      "body": "Your match is scheduled at Court 1 at 14:00"
    },
    "data": {
      "matchId": "m_832918",
      "type": "MATCH_SCHEDULED"
    }
  }
}
```

Listing 1.3: FCM Adapter Contract Payload

1.10.3 APNs Adapter Contract Payload

```
{
  "aps": {
    "alert": {
      "title": "Match Scheduled",
      "body": "Your match is scheduled at Court 1 at 14:00"
    },
    "badge": 1,
    "sound": "default"
  },
  "matchId": "m_832918",
  "type": "MATCH_SCHEDULED"
}
```

Listing 1.4: APNs Adapter Contract Payload

1.11 Technical Stack Overview

The technology stack is selected to prioritize robust horizontal scaling, absolute compile-time type safety, operational simplicity, and modern offline accessibility.

1.11.1 Backend Runtime & Application Layer

- **Runtime**: **Node.js LTS (v20)** + **TypeScript v5**. Provides structural typing, fast startup performance, and native asynchronous scheduling natively suited for non-blocking networking.
- **Application Framework**: **NestJS v10**. Utilizes structured module interfaces, robust dependency injection, architectural patterns, and direct compatibility with Express and Socket.io gateways.

1.11.2 Database & Persistence Strategies

- **Relational Database**: **PostgreSQL v16**. Enforces atomic state machine transitions, complex cross-module query integrity where needed (by abstraction layers), and strong structural indices. Handles the `EloLedger` operations safely under high transactional isolation levels.
- **Object-Relational Mapping (ORM)**: **TypeORM v0.3**. TypeORM supports decorator-driven data modeling, repository patterns, migrations, and direct integration with NestJS databases.

1.11.3 Caching, Realtime, & Queue Management

- **In-Memory Store & Cache**: **Redis v7**. Used for session mapping, active matchmaking ticket indexes, lock primitives, and distributed pub/sub communication channels for scaling horizontal Socket.io servers.
- **Message Broker & Job Queue**: **BullMQ v5**. Integrates with Redis to implement delayed execution jobs, persistent queue processing, backoff retry limits, and distributed scheduling for tournaments and matchmaking cycles.
- **Realtime Communication Gateway**: **Socket.io v4**. Selected for standard WebSocket connectivity, heartbeat checks, automatic network reconnection algorithms, and room isolation groupings (e.g., match chats, live scoring lobbies).

1.11.4 Authentication & Security Infrastructure

- **Token Implementation**: JWT stateless access tokens with short lifetimes (15 minutes). Signed with RSA-256 private keys.
- **Session Persistence**: Long-lived refresh tokens stored inside secure, client-inaccessible, `HttpOnly`, `Secure`, `SameSite=Strict` cookie headers, protecting credentials against Cross-Site Scripting (XSS) and Session Hijacking.
- **Access Control**: Role-Based Access Control (RBAC) integrated into NestJS application controllers using routing guards.

1.11.5 Frontend Framework & Application Packaging

- **Client Runtime**: **React v18** + **TypeScript**. Structured component composition and state synchronization using functional logic.
- **Build Tooling**: **Vite v5**. Optimized hot reloading, standard module splitting, and rapid build processes.
- **Global Client State**: **Zustand**. Lightweight store architecture with zero context-re-render issues.
- **Server Cache Sync**: **TanStack Query v5** (React Query). Governs backend HTTP query lifecycles, automated refetches, and state synchronization.
- **Native Mobile Adapter**: **CapacitorJS v5**. Bundles React web assets into native Android and iOS applications, exposing platform hooks (push channels, lifecycle synchronization adapters) directly to the web runtime.

1.11.6 Infrastructure & Deployments

- **Deployment Model**: Monolithic containers running inside AWS Elastic Container Service (ECS) on AWS Fargate.
- **CDN & Edge WAF**: Cloudflare. Manages standard Static Page distribution, DDoS shielding, and SSL/TLS edge routing.
- **Database Management**: Amazon RDS PostgreSQL with Multi-AZ replications.
- **Caching Management**: Amazon ElastiCache Redis.

Chapter 2

Complete OOP Domain Layer (No Truncation)

Below are the complete, translated, and detailed class architectures from the V1 baseline specification. All variables, types, and logic are fully abstract and tech-stack-agnostic.

2.1 Domain 1: Identity, RBAC & Teams

Manages security identities, Courses, Institutes, user preference presets, and teams.

```
class User {
    id: UUID;
    nusp: String;
    nickname: String;
    email: String;
    fullName: String;
    birthDate: Date;
    pinHash: String; // Securely hashed 4-digit numerical PIN
    avatarUrl: String; // Optional avatar image URL
    instituteId: UUID;
    courseId: UUID;
    availabilityStatus: AvailabilityStatus; // AVAILABLE, MATCHED, OFFLINE
    isDeleted: Boolean;
}

class PublicUserDTO {
    id: UUID;
    nickname: String;
    instituteId: UUID;
    courseId: UUID;
}

class PrivateUserDTO {
    nusp: String;
    email: String;
    fullName: String;
    birthDate: Date;
}

class Institute {
    id: UUID;
    name: String;
    description: String;
}

class Course {
```

```
    id: UUID;
    name: String;
    description: String;
}

class Role {
    id: UUID;
    name: String;
    description: String;
}

class UserRole {
    userId: UUID;
    roleId: UUID;
}

class UserPreferences {
    userId: UUID;
    allowPushNotifs: Boolean;
    muteLobbyChat: Boolean;
    theme: String;
    soundEffectsEnabled: Boolean;
    vibrationEnabled: Boolean;
}

class DeviceToken {
    userId: UUID;
    tokenString: String;
    platform: String; // IOS, ANDROID
    lastUsedAt: DateTime;
}

abstract class Team {
    id: UUID;
    name: String;
    captainId: UUID;
    createdAt: DateTime;
}

class OfficialTeam extends Team {
    instituteId: UUID;
    isActiveCompetitionTeam: Boolean;
}

class TemporaryEventTeam extends Team {
    associatedEventId: UUID;
    expiresAt: DateTime;
}

class TeamRoster {
    teamId: UUID;
    userId: UUID;
    joinedAt: DateTime;
    status: RosterStatus; // ACTIVE, INVITATION_PENDING, REMOVED
    seedNumber: Integer;
}
```

Listing 2.1: Identity and Teams Class Definitions

2.2 Domain 2: Facilities, Maintenance & Equipment

Coordinates physical resources, checkouts, and maintenance processes.

```
class PlayArea {
    id: UUID;
    name: String;
    supportedGameTypes: List<GameType>; // Admin-configured mapping (e.g., ['BOLA_8', 'SNOOKER'])
    status: PlayAreaStatus; // EMPTY, IN_USE, MAINTENANCE
    isVirtual: Boolean; // True for transient, dynamically spawned card game spaces (TRUCO, BURACO)
}

class PlayAreaReservation {
    id: UUID;
    playAreaId: UUID;
    userId: UUID;
    scheduledStartTime: DateTime;
    scheduledEndTime: DateTime;
    bufferPaddingMinutes: Integer; // In minutes, to absorb scheduling shifts
    status: ReservationStatus; // CONFIRMED, ACTIVE, COMPLETED, CANCELLED
    version: Integer;
}

class Equipment {
    id: UUID;
    name: String;
    serialNumber: String;
    type: String; // CUE, PADDLE, BALL_SET, CARD_DECK
    status: EquipmentStatus; // AVAILABLE, IN_USE, MAINTENANCE
    description: String;
}

class CheckoutRecord {
    id: UUID;
    equipmentId: UUID;
    userId: UUID;
    playAreaReservationId: UUID; // Optional, null if checked out without reservation association
    checkedOutAt: DateTime;
    returnedAt: DateTime;
    status: CheckoutStatus; // ACTIVE, RETURNED, OVERDUE
    returnCondition: ReturnCondition; // PRISTINE, WORN, DAMAGED
    associatedMaintenanceTicketId: UUID;
}

class MaintenanceTicket {
    id: UUID;
    resourceType: String; // PLAY_AREA, EQUIPMENT
    resourceId: UUID;
    reportedById: UUID;
    description: String;
    reportedAt: DateTime;
    resolvedAt: DateTime;
    status: MaintenanceStatus; // OPEN, INVESTIGATING, RESOLVED
}
```

Listing 2.2: Facilities and Equipment Class Definitions

2.3 Domain 3: Matchmaking, Concurrency & Core Engine

This module contains the matching engine, live gameplay tracking, and physical allocation bounds.

2.3.1 Forfeit Score Semantics

To prevent structural null violations and numerical division-by-zero errors in ELO calculations, the domain models enforce a fallback scoring policy for abrupt match dropouts (e.g., when a heartbeat timeout triggers). The winning participant is credited with a default maximum score (such as 11 or 2), whereas the forfeiting side is locked at 0.

```
abstract class Game {
    id: UUID;
    name: String;
    gameType: GameType;
    minPlayers: Integer;
    maxPlayers: Integer;
    equipmentRequirements: List<EquipmentType>;
}

class TableGame extends Game {
    validatePhysicalBounds(): Boolean {
        // Dynamically tallies operational PlayArea records matching this game type
        // whose statuses are not flagged as MAINTENANCE to verify capacity bounds.
        return true;
    }
}

class CardGame extends Game {
    deckCount: Integer;
    requiresDealer: Boolean;
}

class MatchmakingTicket {
    id: UUID;
    userId: UUID;
    teamId: UUID; // Optional, null if individual matchmaking
    eloRating: Integer;
    gameType: GameType;
    joinedAt: DateTime;
    expiryTime: DateTime;
    status: TicketStatus; // WAITING, MATCHED, CANCELLED, EXPIRED
}

class Matchmaker {
    // Singleton controller logic to match waiting tickets within ELO ranges
    activeTickets: List<MatchmakingTicket>;

    matchmake(): List<Match> {
        // Core matchmaking algorithm: groups compatible tickets
        // Calculates ticket age (currentTime - MatchmakingTicket.joinedAt)
        // and scales open ELO range dynamically to minimize queues.
        // Defensively verifies (currentTime < ticket.expiryTime) before pairing.
        return new List<Match>();
    }
}

class Match {
    id: UUID;
    playAreaReservationId: UUID; // Optional, null for virtual card games
    gameType: GameType;
    status: MatchStatus; // PENDING_RESOURCE_ALLOCATION, IN_PROGRESS, COMPLETED, DISPUTED, CANCELLED
    startedAt: DateTime;
```

```
    endedAt: DateTime;
}

class MatchParticipant {
    matchId: UUID;
    userId: UUID;
    teamId: UUID; // Optional, null if individual match
    score: Integer;
    resultStatus: ParticipantResult; // WINNER, LOSER, FORFEITED, DISQUALIFIED
}

class MatchSet {
    id: UUID;
    matchId: UUID;
    setNumber: Integer;
    scoreSide1: Integer;
    scoreSide2: Integer;
    winnerId: UUID;
}

class MatchEvent {
    id: UUID;
    matchId: UUID;
    userId: UUID; // Instigator of the event
    eventType: String; // Ex: BALL_POTTED, FOUL, GAME_START
    details: String;
    timestamp: DateTime;
}
```

Listing 2.3: Matchmaking and Core Gameplay Class Definitions

2.4 Domain 4: Custom Events, Parties & Tournaments

Manages campus competitions, elimination stages, and match scoreboards.

```
class Event {
    id: UUID;
    name: String;
    creatorId: UUID;
    description: String;
    startTime: DateTime;
    endTime: DateTime;
    status: EventStatus; // PLANNING, ACTIVE, COMPLETED
}

class EventScore {
    eventId: UUID;
    userId: UUID;
    scoreValue: Integer;
    lastUpdatedAt: DateTime;
}

class Tournament {
    id: UUID;
    name: String;
    gameId: UUID;
    format: TournamentFormat; // SINGLE_ELIMINATION, DOUBLE_ELIMINATION,
    ROUND_ROBIN
    registrationStartTime: DateTime;
    registrationEndTime: DateTime;
    status: TournamentStatus; // REGISTRATION, ACTIVE, CONCLUDED
}
```

```
class TournamentMatchSlot {
    id: UUID;
    tournamentId: UUID;
    roundNumber: Integer;
    matchId: UUID;
    parentSlotId: UUID; // Null for final round or Round-Robin stage
}

class GroupStanding {
    tournamentId: UUID;
    teamId: UUID;
    points: Integer;
    matchesWon: Integer;
    matchesLost: Integer;
    scoreDifferential: Integer;
}
```

Listing 2.4: Events and Tournaments Class Definitions

2.5 Domain 5: Academics & Progression / ELO Engine

Manages seasons, practice tracking, player rankings, and immutable ELO records.

```
class Season {
    id: UUID;
    name: String;
    startTime: DateTime;
    endTime: DateTime;
    isActive: Boolean;
}

class PracticeSession {
    id: UUID;
    playAreaId: UUID;
    startedAt: DateTime;
    endedAt: DateTime;
}

class PracticeAttendance {
    practiceSessionId: UUID;
    userId: UUID;
    checkInTime: DateTime;
}

class PlayerRanking {
    id: UUID;
    seasonId: UUID;
    userId: UUID;
    teamId: UUID; // Optional, null if individual ladder profile
    gameType: GameType;
    eloValue: Integer;
    gamesPlayed: Integer;
    lastMatchAt: DateTime;
}

class EloLedger {
    id: UUID;
    userId: UUID;
    teamId: UUID; // Optional, null if individual ladder transaction
    matchId: UUID;
    seasonId: UUID;
```

```
    oldRating: Integer;
    newRating: Integer;
    changeAmount: Integer;
    calculatedAt: DateTime;
}
```

Listing 2.5: Progression and ELO Class Definitions

2.6 Domain 6: Social, Moderation & Telemetry

Governs relationships, chats, reports, security bans, and performance monitoring.

```
class Friendship {
    userId1: UUID;
    userId2: UUID;
    establishedAt: DateTime;
    status: FriendshipStatus; // PENDING, ACCEPTED, BLOCKED
}

class ChatChannel {
    id: UUID;
    type: ChannelType; // PRIVATE_MESSAGE, LOBBY, MATCH_ROOM
    associatedResourceId: UUID; // Null if PM, matchId if MATCH_ROOM
}

class ChatMessage {
    id: UUID;
    channelId: UUID;
    senderId: UUID;
    content: String;
    sentAt: DateTime;
}

class MatchDispute {
    id: UUID;
    matchId: UUID;
    raisedById: UUID;
    reason: String;
    status: DisputeStatus; // UNDER_REVIEW, RESOLVED, DISMISSED
    resolvedAt: DateTime;
    resolvedById: UUID;
    resolutionNotes: String;
}

class GenericReport {
    id: UUID;
    reportedByUserId: UUID;
    targetUserId: UUID;
    reason: String;
    details: String;
    reportedAt: DateTime;
    status: ReportStatus; // OPEN, INVESTIGATING, CLOSED
}

class UserSanction {
    id: UUID;
    userId: UUID;
    type: SanctionType; // WARNING, TEMP_BAN, PERMANENT_BAN
    reason: String;
    startedAt: DateTime;
    expiresAt: DateTime;
    isActive: Boolean;
```

```
    createdById: UUID;
}

class SystemTelemetry {
    id: UUID;
    metricName: String; // Ex: MatchmakingQueueSize, APIResponseTime
    metricValue: Double;
    recordedAt: DateTime;
}
```

Listing 2.6: Social and Telemetry Class Definitions

2.7 Utility & Core Debugging Utilities

Aids system troubleshooting and game lifecycle simulations.

```
class StateDebugger {
    public dumpSystemState(): Map<String, Any> {
        // Collects in-memory queues and active reservations for audit
        return new Map<String, Any>();
    }
}

class MatchSimulatorLogger {
    public logSimulationEvent(matchId: UUID, logMessage: String): void {
        // Records dry-run matchmaking flows to debug calculations
    }
}
```

Listing 2.7: System Debugger Class Definitions

Chapter 3

Operational JSON & Field Mappings

3.1 JSON Showcase: 8-Ball Pool (Bola 8) Scenario

To validate the conceptual domain model against real-world operations, the following JSON snippet exemplifies a complete **8-Ball Pool (Bola 8)** match scenario.


```
{
  "User": {
    "id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "nusp": "12345678",
    "nickname": "PoolKing88",
    "email": "poolking@usp.br",
    "fullName": "Joao Silva",
    "birthDate": "2002-05-15",
    "instituteId": "b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e",
    "courseId": "c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f",
    "availabilityStatus": "MATCHED",
    "isDeleted": false
  },
  "PublicUserDTO": {
    "id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "nickname": "PoolKing88",
    "instituteId": "b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e",
    "courseId": "c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"
  },
  "PrivateUserDTO": {
    "nusp": "12345678",
    "email": "poolking@usp.br",
    "fullName": "Joao Silva",
    "birthDate": "2002-05-15"
  },
  "TableGame": {
    "id": "d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a",
    "name": "8-Ball Pool",
    "gameType": "BOLA_8",
    "minPlayers": 2,
    "maxPlayers": 2,
    "equipmentRequirements": ["CUE", "BALL_SET"]
  },
  "PlayArea": {
    "id": "e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b",
    "name": "Billiard Table 01",
    "gameType": "BOLA_8",
    "status": "IN_USE",
    "version": 7
  },
  "PlayAreaReservation": {
    "id": "f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c",
```

```
    "playAreaId": "e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b",
    "userId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "scheduledStartTime": "2026-06-20T18:00:00Z",
    "scheduledEndTime": "2026-06-20T19:00:00Z",
    "bufferPaddingMinutes": 15,
    "status": "ACTIVE",
    "version": 2
  },
  "Equipment": {
    "id": "a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d",
    "name": "Custom Ash 3/4 Billiard Cue",
    "serialNumber": "SN-CUE-ASH-9981",
    "type": "CUE",
    "status": "IN_USE",
    "description": "High-grade imported Ash wood cue with customized joint"
  },
  "CheckoutRecord": {
    "id": "b8c9d0e1-f2a3-4b5c-6d7e-8f9a0b1c2d3e",
    "equipmentId": "a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d",
    "userId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "playAreaReservationId": "f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c",
    "checkedOutAt": "2026-06-20T17:58:00Z",
    "returnedAt": null,
    "status": "ACTIVE",
    "returnCondition": null,
    "associatedMaintenanceTicketId": null
  },
  "MatchmakingTicket": {
    "id": "c9d0e1f2-a3b4-5c6d-7e8f-9a0b1c2d3e4f",
    "userId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "teamId": null,
    "eloRating": 1500,
    "gameType": "BOLA_8",
    "joinedAt": "2026-06-20T17:45:00Z",
    "expiryTime": "2026-06-20T17:55:00Z",
    "status": "MATCHED"
  },
  "Match": {
    "id": "d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5a",
    "playAreaReservationId": "f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c",
    "gameType": "BOLA_8",
    "status": "IN_PROGRESS",
    "startedAt": "2026-06-20T18:02:00Z",
    "endedAt": null
  },
  "TeamRoster": {
    "teamId": "d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5b",
    "userId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "joinedAt": "2026-06-20T17:00:00Z",
    "status": "ACTIVE",
    "seedNumber": 1
  }
}
```

Listing 3.1: Bola 8 Conceptual JSON Instance

3.2 Completeness Verification Matrix

Table 3.1 registers all system domain classes, detailing their main purposes, core functions, expected lifecycles, and verification check status.

Table 3.1: Completeness Verification Matrix

Class Name	Abstract Purpose	Key Methods	Lifecycle States	Validation
`User`	Security identity profile	`updateStatus()`	AVAILABLE, MATCHED, OFFLINE	Verified
`PlayArea`	Coordinates game tables	`validatePhysicalBounds()`	EMPTY, IN_USE, MAINTENANCE	Verified
`PlayAreaReservation`	Manages slot bookings with padding	`confirmReservation()`	CONFIRMED, ACTIVE, CANCELLED	Verified
`Equipment`	Handles physical checkouts	`updateStatus()`	AVAILABLE, IN_USE, MAINTENANCE	Verified
`CheckoutRecord`	Controls cue loans linked to reservation	`flagOverdue()`	ACTIVE, RETURNED, OVERDUE	Verified
`MatchmakingTicket`	Active queue ticket	`isExpired()`	WAITING, MATCHED, EXPIRED	Verified
`Match`	Regulates side-agnostic progressions	`processSetOutcome()`	PENDING_RESOURCE_ALLOCATION, IN_PROGRESS, COMPLETED	Verified
`TournamentMatchSlot`	Structures tournament slots	`advanceWinner()`	PENDING, READY, COMPLETED	Verified
`EloLedger`	Tracks ranking changes	`recordEloChange()`	COMPLETED	Verified
`GroupStanding`	Handles group-stage standings	`calculateStanding()`	ACTIVE, CONCLUDED	Verified
`PublicUserDTO`	Filters non-sensitive user data	`serialize()`	ACTIVE	Verified
`PrivateUserDTO`	Holds sensitive RBAC user data	`serialize()`	ACTIVE	Verified

Chapter 4

Production Stack Adapter Specifications

This chapter details the production-grade technology implementation of external adapter layers. All components implement the abstract ports defined in Chapter 2, separating core business logic from framework concerns.

4.1 Core Runtime & Backend Framework

4.1.1 Runtime & Language: TypeScript on Node.js / Bun

The application backend executes **TypeScript** compiled to a native runtime on either **Node.js** (LTS) or **Bun** (for environments prioritizing cold-start performance and minimal memory footprint). The central rationale is compile-time type fidelity: the abstract domain interfaces defined throughout the OOP class hierarchy (e.g., `IEnrollPlayerUseCase`, `IPlayAreaRepositoryPort`, `INotificationServicePort`) are expressed as TypeScript `interface` or `abstract class` constructs. This means the runtime carries zero ambiguity at the boundary between core domain logic and its concrete adapters.

```
// Output Port (abstract, lives inside Core Domain)
interface IPlayAreaRepositoryPort {
  findById(id: string): Promise<PlayArea | null>;
  updateWithOptimisticLock(
    area: PlayArea,
    expectedVersion: number
  ): Promise<void>;
}

// Concrete Adapter (lives in Infrastructure layer)
class TypeOrmPlayAreaAdapter implements IPlayAreaRepositoryPort {
  async findById(id: string): Promise<PlayArea | null> { ... }
  async updateWithOptimisticLock(
    area: PlayArea,
    expectedVersion: number
  ): Promise<void> { ... }
}
```

Listing 4.1: TypeScript abstract port interface mapping

Because TypeScript enforces structural typing at compile time, attempting to pass a concrete adapter that does not satisfy the port contract becomes a build error, not a runtime exception. This gives the Hexagonal boundary a hard enforcement mechanism that a purely runtime language cannot replicate without extensive reflection overhead.

4.1.2 Application Framework: NestJS with Strict Modular Architecture

NestJS is selected as the application framework. Its module system maps directly to the six business domain capabilities defined in Section 2, producing one NestJS `Module` per domain: `IdentityModule`, `FacilitiesModule`, `MatchmakingModule`, `TournamentsModule`, `ProgressionModule`, and `SocialModule`.

The critical architectural guarantee is that NestJS's dependency injection (DI) container enforces the Hexagonal separation by using **custom injection tokens** rather than concrete class references in controller constructors. This prevents framework-layer controllers from importing or instantiating domain use cases directly.

```
// tokens.ts (shared constants, imported by module providers)
export const ENROLL_PLAYER_USE_CASE
  = Symbol('IEnrollPlayerUseCase');

// matchmaking.module.ts (Infrastructure/Framework layer)
@Module({
  providers: [
    {
      provide: ENROLL_PLAYER_USE_CASE,
      useClass: EnrollPlayerUseCase, // concrete application service
    },
    TypeOrmMatchmakingTicketAdapter,
    RedisMatchmakingQueueAdapter,
  ],
  controllers: [MatchmakingHttpController],
})
export class MatchmakingModule {}

// matchmaking-http.controller.ts
@Controller('/matchmaking')
export class MatchmakingHttpController {
  constructor(
    @Inject(ENROLL_PLAYER_USE_CASE)
    private readonly enrollPlayer: IEnrollPlayerUseCase,
  ) {}

  @Post('/queue')
  async joinQueue(@Body() dto: EnrollPlayerDto) {
    return this.enrollPlayer.execute(dto);
  }
}
```

Listing 4.2: NestJS Hexagonal DI Token Pattern

The HTTP controller imports only the `IEnrollPlayerUseCase` interface type and the symbol token. The concrete `EnrollPlayerUseCase` class is resolved entirely by the NestJS container at runtime. A controller swap (e.g., replacing REST with gRPC) requires no changes inside the core domain.

4.2 Persistence & In-Memory Caching Layer

4.2.1 Relational Storage: PostgreSQL with TypeORM

PostgreSQL is the ACID-compliant relational engine. It is accessed via **TypeORM** using the `DataMapper` pattern, deliberately keeping entity annotations inside the Infrastructure layer and away from the pure domain classes. The following table maps core domain classes to their physical PostgreSQL table representations:

Domain Class	PG Table	Key Constraints
User	users	UNIQUE(nusp), UNIQUE(email), FK(institute_id), FK(course_id)
PlayAreaReservation	play_area_reservations	FK(play_area_id), CHECK(end > start), game_type VARCHAR, version INTEGER
EloLedger	elo_ledger	FK(user_id, match_id, season_id), IMMUTABLE rows
GroupStanding	group_standings	COMPOSITE PK(tournament_id, team_id)
CheckoutRecord	checkout_records	FK(equipment_id, user_id), FK(reservation_id) NULLABLE
UserSanction	user_sanctions	FK(user_id), INDEX(is_active, expires_at)

4.2.2 Optimistic Concurrency Control for the Bola 8 Table

The single physical billiard table imposes the strictest concurrency constraint in the entire system: at most one `PlayAreaReservation` may hold the `ACTIVE` status for a given `PlayArea` at any instant. TypeORM's `@VersionColumn` decorator implements the abstract OCC policy defined in Section 6.1:

```
// Infrastructure layer -- TypeORM entity mapper
@Entity('play_areas')
export class PlayAreaEntity {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'varchar' })
  status: string; // EMPTY | IN_USE | MAINTENANCE

  @VersionColumn()
  version: number; // maps to abstract version: Integer field
}

// In the adapter: reserve table with OCC guard
async reservePlayArea(
  areaId: string,
  expectedVersion: number
): Promise<void> {
  const result = await this.repo
    .createQueryBuilder()
    .update(PlayAreaEntity)
    .set({ status: 'IN_USE', version: () => 'version + 1' })
    .where('id = :id AND version = :v', {
      id: areaId, v: expectedVersion
    })
    .execute();

  if (result.affected === 0) {
    // Version mismatch: another thread already committed
    throw new OptimisticLockException(areaId);
  }
}
```

Listing 4.3: TypeORM OCC decorator on PlayArea entity

When `OptimisticLockException` propagates, the application use case intercepts it and immediately re-queues the paired match participants at the front of the matchmaking queue (imple-

menting the "Matched but Homeless" fallback described in Section 6.1), preserving their queue priority without exposing the conflict detail to the client.

The transaction isolation level for all reservation writes is configured globally:

```
// TypeORM data source configuration
{
  type: 'postgres',
  isolationLevel: 'REPEATABLE READ',
  // SERIALIZABLE used for reservation slot conflict writes
}

// Per-operation override for critical reservation paths
await dataSource.transaction(
  'SERIALIZABLE',
  async (manager) => {
    await manager.findOne(PlayAreaEntity, { ... });
    await manager.save(ReservationEntity, { ... });
  }
);
```

Listing 4.4: PostgreSQL transaction isolation configuration

4.2.3 In-Memory Caching: Redis

Redis serves the volatile state layer described in Section 6.2. Three key namespace families are defined:

Matchmaking Queue Sorted Sets Each campus-isolated queue is stored as a Redis `ZSET`. The sort score is the Unix timestamp of `MatchmakingTicket.joinedAt`, so earlier entries are naturally dequeued first. Re-queued "Matched but Homeless" players are re-inserted with a score of 0 to guarantee absolute front priority:

```
# Namespace pattern
gamehub:{campus_id}:queue:{game_type}
# Example key
gamehub:campus_sp:queue:bola_8

# Add ticket to queue
ZADD gamehub:campus_sp:queue:bola_8
      <joinedAt_unix_ms> <ticketId>

# Re-queue at front (Matched-but-Homeless recovery)
ZADD gamehub:campus_sp:queue:bola_8 0 <ticketId>

# Dequeue N oldest tickets for matchmaking
ZPOPMIN gamehub:campus_sp:queue:bola_8 2
```

Listing 4.5: Redis queue namespace and operations

Active Ticket Duplicate Lock To prevent a user submitting a second ticket while one is already `WAITING`, a Redis String key is written atomically using `SET NX EX`:

```
# Key pattern
gamehub:ticket_lock:{userId}:{game_type}

# Atomic set-if-not-exists with TTL matching expiryTime
SET gamehub:ticket_lock:a1b2c3d4:bola_8
    <ticketId> NX EX 600
# Returns nil if a ticket already exists for this user
```

Listing 4.6: Duplicate ticket guard O(1) lookup

Heartbeat Presence Keys Client heartbeat presence is tracked via volatile keys with a 15-second TTL. The BullMQ worker listens for keyspace expiry events to trigger forfeiture flows:

```
# Key pattern
gamehub:heartbeat:{userId}

# Client sends heartbeat every 5 seconds
SET gamehub:heartbeat:a1b2c3d4 1 EX 15

# Keyspace expiry event consumed by BullMQ worker
# triggers: forfeit active match, free table reservation
```

Listing 4.7: Heartbeat key TTL namespace

4.3 Real-Time Network & Streaming Transport

4.3.1 WebSocket Gateway: NestJS + Socket.io

Socket.io (backed by Engine.io transport negotiation) is integrated via `@nestjs/websockets`. The `@WebSocketGateway` decorator creates the Input Adapter for the real-time boundary, bridging incoming WebSocket frames to core domain use cases through the same injection token pattern used by HTTP controllers.

Match-specific rooms isolate event broadcasts. When a `Match` transitions to `IN_PROGRESS`, both participants are joined to a room keyed by the match UUID. Ball-potting events (e.g., `MatchEvent.eventType = BALL_POTTED`) are emitted only to members of that room:

```
@WebSocketGateway({ namespace: '/match', cors: true })
export class MatchGateway {
  @WebSocketServer() server: Server;

  @SubscribeMessage('report_score')
  async handleScoreReport(
    @ConnectedSocket() client: Socket,
    @MessageBody() payload: ScoreReportDto,
  ) {
    const result = await this.recordSetScore.execute(payload);
    // Broadcast only to match room
    this.server
      .to(`match:${payload.matchId}`)
      .emit('score_updated', result);
  }

  // Client presence heartbeat handler
  @SubscribeMessage('heartbeat')
  async handleHeartbeat(@ConnectedSocket() client: Socket) {
    const userId = client.data.userId;
    await this.redis.set(
      `gamehub:heartbeat:${userId}`, '1', 'EX', 15
    );
  }
}
```

Listing 4.8: WebSocket gateway room and event pattern

When a heartbeat key expires in Redis (TTL reaches zero), a Keyspace Notification triggers the BullMQ worker (see Section 10.4), which flags the player as offline and issues the forfeit.

4.3.2 REST API Transport: Fastify Adapter

NestJS is bound to the **Fastify** HTTP engine via `FastifyAdapter` for all REST endpoints, replacing the default Express adapter. Fastify's zero-overhead routing and Ajv-based JSON

schema validation reduce request-processing latency for high-frequency endpoints such as the matchmaking queue entry call.

The standardized error envelope defined in Section 7.4 is enforced globally through a NestJS **Exception Filter**:

```
@Catch()
export class GlobalExceptionFilter
    implements ExceptionFilter {

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<FastifyReply>();

    const status = exception instanceof HttpException
      ? exception.getStatus()
      : 500;

    const errorCode = exception instanceof DomainException
      ? exception.code
      : 'INTERNAL_ERROR';

    // Standardized envelope matching Section 7.4 schema
    response.status(status).send({
      errorCode,
      displayMessage: (exception as Error).message,
      timestamp: new Date().toISOString(),
      details: exception instanceof DomainException
        ? exception.details
        : {},
    });
  }
}
```

Listing 4.9: Global exception filter enforcing error envelope

4.3.3 Native Push Notification Output Adapters: FCM & APNs

Beyond in-channel WebSocket broadcasts, the Game Hub must reach users who are offline or have the app minimized. Two concrete output adapters realize the abstract `INotificationServicePort` contract defined in Section 5.1:

- **FirebaseCloudMessagingAdapter** (FCM): targets Android devices and Chrome/web progressive-app endpoints.
- **ApplePushNotificationServiceAdapter** (APNs): targets iOS devices using p8-key-authenticated HTTP/2 connections.

Both classes live strictly inside the Infrastructure layer and are injected via the same token pattern used by repository adapters:

```
// Abstract port (Core Domain -- Section 5.1)
interface INotificationServicePort {
  sendPush(
    token: string,
    platform: 'IOS' | 'ANDROID',
    payload: PushPayload
  ): Promise<void>;
}

// Infrastructure: FCM adapter
class FirebaseCloudMessagingAdapter
```

```
    implements INotificationServicePort {

  async sendPush(
    token: string,
    platform: 'IOS' | 'ANDROID',
    payload: PushPayload
  ): Promise<void> {
    await this.fcmAdminSdk.messaging().send({
      token,
      // data-only (silent) payload for background wakeup
      data: {
        event: payload.event,
        matchId: payload.matchId ?? '',
        priority: payload.priority,
      },
      android: { priority: 'high' },
    });
  }
}

// Infrastructure: APNs adapter
class ApplePushNotificationServiceAdapter
    implements INotificationServicePort {

  async sendPush(
    token: string,
    platform: 'IOS' | 'ANDROID',
    payload: PushPayload
  ): Promise<void> {
    await this.apn.send({
      deviceToken: token,
      // content-available: 1 triggers silent background fetch
      contentAvailable: true,
      payload: { event: payload.event,
                 matchId: payload.matchId },
    });
  }
}
```

Listing 4.10: FCM and APNs adapter classes implementing INotificationServicePort

The `DeviceToken` domain class (Section 3.1) is persisted in a dedicated PostgreSQL table so that adapters can resolve the correct token and platform for any given user:

```
-- Table: device_tokens
CREATE TABLE device_tokens (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(id)
                 ON DELETE CASCADE,
  token_string TEXT NOT NULL UNIQUE,
  platform     VARCHAR(8) NOT NULL CHECK
                 (platform IN ('IOS', 'ANDROID')),
  last_used_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Index for fast per-user token lookups
CREATE INDEX idx_device_tokens_user
  ON device_tokens(user_id);
```

Listing 4.11: DeviceToken PostgreSQL table schema

The NestJS DI token `NOTIFICATION_SERVICE_PORT` resolves the correct adapter at runtime based on the environment configuration, keeping the core domain unaware of which vendor is active.

4.4 Background Orchestration & Asynchronous Workers

4.4.1 Distributed Task Queue: BullMQ on Redis Streams

BullMQ manages all background asynchronous workers, using Redis Streams as its persistence backbone. Each asynchronous concern from Section 6.3 maps to a named BullMQ queue:

BullMQ Queue Name	Responsibility
`ticket-expiry-scan`	Scans `WAITING` tickets past `expiryTime` and transitions them to `EXPIRED`; purges queue entries.
`heartbeat-timeout-handler`	Reacts to Redis keyspace TTL expiry; triggers player offline flag, match forfeit, and table reservation cancellation.
`match-chat-eviction`	Transitions match `ChatChannel` to read-only when `MatchStatus` becomes `COMPLETED` or `CANCELLED`.
`no-show-monitor`	Cancels `CONFIRMED` reservations that exceed their start time by a configurable threshold without activation.
`sanction-cascade-enforcer`	Intercepts `UserSanction` activation; overrides `availabilityStatus`, purges tickets and reservations.

Each worker is encapsulated in a NestJS `Processor` class, consuming jobs from its designated queue:

```
@Processor('heartbeat-timeout-handler')
export class HeartbeatTimeoutProcessor
    extends WorkerHost {

  async process(job: Job<{ userId: string }>): Promise<void> {
    const { userId } = job.data;

    // 1. Set availability to OFFLINE
    await this.userRepo.updateStatus(userId, 'OFFLINE');

    // 2. Cancel active MatchmakingTickets
    await this.ticketRepo.cancelActiveByUser(userId);

    // 3. Apply forfeit to any in-progress match
    const activeMatch =
      await this.matchRepo.findActiveByUser(userId);
    if (activeMatch) {
      await this.forfeitUseCase.execute({
        matchId: activeMatch.id,
        forfeitingUserId: userId,
      });
    }

    // 4. Release PlayArea reservation
    await this.reservationRepo
      .cancelUpcomingByUser(userId);

    // 5. Dispatch silent push via FCM/APNs Output Adapter
    //    to alert the mobile device immediately
    const tokens = await this.deviceTokenRepo
      .findByUser(userId);
    for (const dt of tokens) {
      await this.notificationPort.sendPush(
        dt.tokenString,
        dt.platform,
```

```
        { event: 'HEARTBEAT_EXPIRED',
          matchId: activeMatch?.id,
          priority: 'HIGH' }
      );
    }
  }
}
```

Listing 4.12: BullMQ heartbeat timeout processor

The `sanction-cascade-enforcer` worker follows the same pattern, dispatching a high-priority silent push payload after banning a user to ensure the mobile app reacts even when backgrounded:

```
@Processor('sanction-cascade-enforcer')
export class SanctionCascadeProcessor extends WorkerHost {

  async process(
    job: Job<{ userId: string; sanctionId: string }>
  ): Promise<void> {
    const { userId } = job.data;

    await this.userRepo.updateStatus(userId, 'OFFLINE');
    await this.ticketRepo.cancelActiveByUser(userId);
    await this.reservationRepo.cancelUpcomingByUser(userId);

    const tokens = await this.deviceTokenRepo
      .findByUser(userId);
    for (const dt of tokens) {
      await this.notificationPort.sendPush(
        dt.tokenString, dt.platform,
        { event: 'ACCOUNT_SANCTIONED', priority: 'HIGH' }
      );
    }
  }
}
```

Listing 4.13: Sanction cascade worker with push dispatch

4.4.2 Redis Keyspace Notifications as Worker Triggers

Redis is configured to emit **keyspace expiry notifications** (`notify-keyspace-events KEA`). The BullMQ Redis connection subscribes to the `__keyevent@0__:expired` channel. When a `gamehub:heartbeat:{userId}` key expires, the subscriber enqueues a job in the `heartbeat-timeout-handler` queue:

```
# Redis server config
notify-keyspace-events KEA

# Application subscriber (pseudo-code)
redisSubscriber.subscribe('__keyevent@0__:expired')
redisSubscriber.on('message', (channel, key) => {
  if (key.startsWith('gamehub:heartbeat:')) {
    const userId = key.split(':')[2];
    heartbeatQueue.add('timeout', { userId });
  }
});
```

Listing 4.14: Redis keyspace notification subscriber

This event-driven design eliminates polling loops entirely. The heartbeat worker is only invoked when a real expiry occurs, keeping CPU overhead proportional to actual disconnection events. Each expiry also chains a native push dispatch through the `INotificationServicePort`

adapters, ensuring mobile devices receive an immediate alert regardless of WebSocket channel state.

Chapter 5

Client-Side State Engine

This chapter defines the client-side state transitions, connectivity handling, and native mobile container integration.

5.1 Frontend & Client-Side Architecture

5.1.1 User Interface: React with TypeScript

The client-side interface is built with **React** using TypeScript. Component boundaries are structured to mirror the domain capabilities: one top-level feature module per domain (e.g., `/features/matchmaking`, `/features/facilities`, `/features/social`). Each feature module contains its own local components, hooks, service calls, and typed DTO definitions, keeping cross-domain coupling to explicit shared types only.

Sensitive data rendering is gated by the `PublicUserDTO` / `PrivateUserDTO` split. Leaderboard and opponent profile views consume only `PublicUserDTO` fields (`id`, `nickname`, `instituteId`, `courseId`). The private profile edit form receives a `PrivateUserDTO` only if the authenticated user's JWT claims match the target profile's UUID, enforced by the REST endpoint's RBAC guard.

5.1.2 Client Connectivity State Engine: Zustand

The five-state connectivity machine (Section 7.2) is implemented as a **Zustand** global store with typed actions. The exponential backoff with randomized jitter is embedded directly into the `reconnect` action:

```
type ConnState =
  | 'OFFLINE' | 'CONNECTING' | 'CONNECTED'
  | 'RECONNECTING' | 'TERMINATED';

interface SocketStore {
  state: ConnState;
  attempt: number;
  connect: () => void;
  onDisconnect: () => void;
  onReconnectSuccess: () => void;
  terminate: () => void;
}

export const useSocketStore =
  create<SocketStore>((set, get) => ({
  state: 'OFFLINE',
  attempt: 0,

  connect: () => {
```

```
    set({ state: 'CONNECTING' });
    socketClient.connect({
      onSuccess: () => {
        set({ state: 'CONNECTED', attempt: 0 });
        startHeartbeatInterval(); // 5-second ping loop
      },
      onFailure: () => set({ state: 'OFFLINE' }),
    });
  },

  onDisconnect: () => {
    const { attempt } = get();
    set({ state: 'RECONNECTING',
          attempt: attempt + 1 });
    const jitter = Math.random() * 1000;
    // Exponential backoff: 2^attempt + random jitter (ms)
    const delay = Math.pow(2, attempt) * 1000 + jitter;
    setTimeout(() => get().connect(), delay);
  },

  onReconnectSuccess: () =>
    set({ state: 'CONNECTED', attempt: 0 }),

  terminate: () =>
    set({ state: 'TERMINATED', attempt: 0 }),
}));
```

Listing 5.1: Zustand connectivity state machine with jitter backoff

The `startHeartbeatInterval` function emits a `heartbeat` WebSocket event every 5 seconds, refreshing the `gamehub:heartbeat:{userId}` TTL in Redis and preventing the BullMQ timeout worker from triggering a false-positive forfeit.

5.1.3 Authentication & Security: JWT with HttpOnly Cookies

Authentication follows a dual-token strategy:

- **Access Token (JWT)**: Short-lived (15 minutes), stored *in client memory only* (JavaScript variable, never `localStorage`). Contains RBAC claims (`roles`, `userId`, `instituteId`).
- **Refresh Token**: Long-lived (7 days), stored exclusively in an **HttpOnly**, **SameSite=Strict** cookie, preventing JavaScript access and mitigating XSS token theft.

```
// JWT payload (decoded in client memory, never persisted)
interface JwtPayload {
  sub: string;          // userId (UUID)
  roles: string[];      // ['STUDENT', 'ADMIN', 'MODERATOR']
  instituteId: string;
  exp: number;
}

// Client-side RBAC visual gate
function AdminMenuGuard({ children }: { children: ReactNode }) {
  const { payload } = useAuthStore();
  const isAdmin = payload?.roles.includes('ADMIN') ?? false;

  // Admin UI element hidden; endpoint still validates server-side
  if (!isAdmin) return null;
  return <>{children}</>;
}

// Refresh flow: sends HttpOnly cookie automatically
```

```
async function refreshAccessToken(): Promise<string> {
  // Cookie is sent automatically by the browser;
  // no JS access to the refresh token value itself.
  const res = await fetch('/auth/refresh', {
    method: 'POST',
    credentials: 'include', // transmit HttpOnly cookie
  });
  const { accessToken } = await res.json();
  return accessToken; // stored in Zustand memory store
}
```

Listing 5.2: JWT claims structure and client-side RBAC check

The NestJS `JwtAuthGuard` validates every incoming REST and WebSocket request by verifying the `Authorization: Bearer <token>` header signature and expiry. Role checks are applied via a `RolesGuard` reading the `@Roles()` decorator on controller methods, ensuring that endpoint-level authorization is never bypassed by client-side visual gating alone.

5.1.4 Hybrid Native Cross-Platform Shell: Capacitor

The React and TypeScript codebase is wrapped into native iOS and Android shells using **Capacitor** (`@capacitor/core` and `@capacitor/cli`). Capacitor embeds the compiled SPA as a WebView, routing its URL to the static asset build output directory:

```
{
  "appId": "br.usp.gamehub",
  "appName": "GameHub",
  "webDir": "dist",
  "bundledWebRuntime": false,
  "server": {
    "hostname": "gamehub.usp.br",
    "androidScheme": "https"
  },
  "plugins": {
    "PushNotifications": {
      "presentationOptions": ["badge", "sound", "alert"]
    }
  }
}
```

Listing 5.3: capacitor.config.json – SPA asset routing

Capacitor plugins required by the mobile shell:

- `@capacitor/push-notifications`: registers the native device with FCM (Android) or APNs (iOS), receives the device token string, and forwards it to the backend `POST /device-tokens` REST endpoint to be persisted in the `device_tokens` table.
- `@capacitor/app`: exposes native lifecycle events (`appStateChange`) to the JavaScript layer.
- `@capacitor-community/secure-storage`: encrypted native key-value store used as a WebKit fallback.

```
import { PushNotifications } from '@capacitor/push-notifications';

async function registerPushToken(
  userId: string
): Promise<void> {
  await PushNotifications.requestPermissions();
  await PushNotifications.register();

  PushNotifications.addListener(
```

```
    'registration',
    async (token) => {
      // Persist token to backend device_tokens table
      await fetch('/device-tokens', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json',
                   'Authorization': `Bearer ${getAccessToken()}` },
        body: JSON.stringify({
          tokenString: token.value,
          platform: Capacitor.getPlatform()
                    === 'ios' ? 'IOS' : 'ANDROID',
        }),
      });
    }
  );
}
```

Listing 5.4: Push token registration via @capacitor/push-notifications

5.1.5 WebKit Secure Storage Cookie Fallback

On iOS, WebKit's Intelligent Tracking Prevention (ITP) and strict cross-origin isolation policies can silently discard `HttpOnly` cookies set by the backend within a Capacitor WebView context. The authentication layer detects this condition and falls back to `@capacitor-community/secure-storage` for Refresh Token persistence, transmitting the token through an explicit `Authorization` header instead of relying on the browser cookie jar.

```
import { SecureStoragePlugin }
  from '@capacitor-community/secure-storage';
import { Capacitor } from '@capacitor/core';

const REFRESH_KEY = 'gamehub_refresh_token';

async function refreshAccessToken(): Promise<string> {
  const isNative = Capacitor.isNativePlatform();

  let body: BodyInit | undefined;
  let headers: HeadersInit = {
    'Content-Type': 'application/json'
  };

  if (isNative) {
    // iOS/Android: read from encrypted native store
    const { value } = await SecureStoragePlugin.get({
      key: REFRESH_KEY
    });
    // Transmit via Authorization header, not cookie
    headers['X-Refresh-Token'] = value ?? '';
  }
  // Web: browser sends HttpOnly cookie automatically
  const res = await fetch('/auth/refresh', {
    method: 'POST',
    credentials: 'include',
    headers,
    body,
  });
  const { accessToken, refreshToken } = await res.json();

  if (isNative && refreshToken) {
    // Persist updated refresh token to native secure store
    await SecureStoragePlugin.set({
      key: REFRESH_KEY, value: refreshToken
```

```
    });
  }
  return accessToken;
}
```

Listing 5.5: WebKit cookie fallback to SecureStorage

The NestJS `/auth/refresh` endpoint is updated to accept the Refresh Token from either the `HttpOnly` cookie or the `X-Refresh-Token` header, preferring the cookie on web and the header on native, with identical signature validation in both paths.

5.1.6 Mobile App Minimization Lifecycle Hook

When the user presses the home button or switches apps, the host OS schedules the WebView's JavaScript thread for suspension within a bounded time window. To prevent this freeze from allowing a stale heartbeat key to expire in Redis and triggering a false-positive forfeit, the Zustand state machine intercepts the native `appStateChange` event from `@capacitor/app` and fires an explicit `minimize_presence` WebSocket frame before execution is frozen.

```
import { App, AppState } from '@capacitor/app';

// Bootstrap: called once at app startup
export function initMobileLifecycleBroker(): void {
  App.addListener(
    'appStateChange',
    (state: AppState) => {
      const { socket } = useSocketStore.getState();

      if (!state.isActive) {
        // App is being minimized.
        // Fire minimize_presence SYNCHRONOUSLY before
        // the OS freezes the JS thread.
        socket?.emit('minimize_presence', {
          userId: useAuthStore.getState().payload?.sub,
          timestamp: Date.now(),
        });
        // Extend the Redis heartbeat TTL proactively
        // to absorb the background freeze window.
        socket?.emit('heartbeat', {});
      } else {
        // App returned to foreground.
        const { state: connState, connect } =
          useSocketStore.getState();
        if (connState === 'OFFLINE'
            || connState === 'RECONNECTING') {
          connect();
        }
      }
    }
  );
}
```

Listing 5.6: Capacitor app lifecycle interceptor in Zustand

On the backend, the `MatchGateway` handles the `minimize_presence` message by refreshing the Redis heartbeat TTL with an extended value (e.g., 60 seconds instead of the standard 15), providing a grace window for the OS background freeze.

```
@SubscribeMessage('minimize_presence')
async handleMinimizePresence(
  @ConnectedSocket() client: Socket,
) {
  const userId = client.data.userId;
```

```
    // Extended TTL: 60 s to cover OS background freeze
    await this.redis.set(
      `gamehub:heartbeat:${userId}`, '1', 'EX', 60
    );
}
```

Listing 5.7: Backend minimize_presence handler in MatchGateway

Chapter 6

Implementation Planning & Delivery Roadmap

This implementation blueprint spans from MVP definition to multi-platform production launch.

6.1 Phase 0: MVP Definition

6.1.1 Objectives

Scope and detail the base functional system to test core flows (identity, simple matchmaking, reservation locking).

6.1.2 Tangible Architectural Deliverables

Comprehensive system specification, entity diagram designs, user story tracking boards.

6.1.3 Strict Technical Dependencies

None.

6.1.4 Measurable Acceptance Criteria

All domain relationship flows agreed upon by team leads. Target metrics defined.

6.1.5 Identified Risks & Mitigation Protocols

Scope Creep: Prevent features extensions. Lock MVP bounds using strict backlog management.

6.1.6 Estimated Implementation Complexity

Low (1/5).

6.2 Phase 1: Foundation (Monorepo, CI/CD, Observability)

6.2.1 Objectives

Establish the monorepo architecture, testing infrastructure, automated deployment pipelines, and centralized metrics monitoring frameworks.

6.2.2 Tangible Architectural Deliverables

Monorepo directory setup, CI pipeline configuration scripts, Prometheus integration setups.

6.2.3 Strict Technical Dependencies

Phase 0 architectural approval.

6.2.4 Measurable Acceptance Criteria

Successful pipeline runs verify code linting, run unit tests, build Docker containers, and emit metric logs.

6.2.5 Identified Risks & Mitigation Protocols

Pipeline Bottlenecks: Long build times. Mitigation: Implement Docker caching layers in GitHub Actions runner cache.

6.2.6 Estimated Implementation Complexity

Medium (3/5).

6.2.7 Operational Deployment Status

The monorepo structure is fully operational using npm workspaces. The CI/CD pipeline is active at `.github/workflows/ci.yml`, incorporating automated build, lint, and test scripts along with multi-stage Docker build steps using the optimized `type=gha` Docker buildx caching provider. Centralized observability bootstrapping is active at `backend/src/shared/telemetry/otel-bootstrap.ts`.

6.3 Phase 2: Core Backend (Auth, RBAC, Users, Teams)

6.3.1 Objectives

Deploy identity management services, role validations, password hashes, and team structures.

6.3.2 Tangible Architectural Deliverables

PostgreSQL database schemas for users/teams, Argon2 hash validation code, JWT signature token verifiers.

6.3.3 Strict Technical Dependencies

Phase 1 foundation monorepo pipelines.

6.3.4 Measurable Acceptance Criteria

API endpoints for authentication yield valid access tokens, enforce route protection rules, and return membership arrays.

6.3.5 Identified Risks & Mitigation Protocols

JWT Key Leakage: Mitigation: Secrets injected via AWS Parameter Store and rotated every 30 days.

6.3.6 Estimated Implementation Complexity

Medium (3/5).

6.3.7 Operational Deployment Status

The identity, authentication, and team management logic is fully implemented under `src/modules/identity/`. The secure 4-digit PIN authentication verification is integrated directly into the rich `User` domain model. Access control is enforced via custom `@Roles` metadata annotations and framework guards (`JwtAuthGuard`, `RolesGuard`). The relational database schema in `schema.sql` has been updated to include indices, `teams`, and `team_rosters` layout mappings.

6.4 Phase 3: Facilities & Reservations

6.4.1 Objectives

Implement play areas mapping, timeslot creation, and reservation locking logic preventing double bookings.

6.4.2 Tangible Architectural Deliverables

Database tables for play areas and reservations, transactional lock code enforcing unique booking allocations.

6.4.3 Strict Technical Dependencies

Phase 2 identity systems.

6.4.4 Measurable Acceptance Criteria

Overlapping timeslots allocations are blocked by the database unique constraints.

6.4.5 Identified Risks & Mitigation Protocols

Resource Contention: High load locking database lines. Mitigation: Limit booking windows to 14 days in advance.

6.4.6 Estimated Implementation Complexity

Medium (3/5).

6.5 Phase 4: Matchmaking & Realtime (Queues, WebSockets, Concurrency)

6.5.1 Objectives

Integrate Redis queues, real-time message routing layers, and websocket connections.

6.5.2 Tangible Architectural Deliverables

Socket.io Gateway interface validation layers, BullMQ queue dispatchers, and worker processors.

6.5.3 Strict Technical Dependencies

Phase 3 play area structures.

6.5.4 Measurable Acceptance Criteria

Simulated WebSocket clients join matchmaking queues, match pairings occur based on rating variables, and events propagate.

6.5.5 Identified Risks & Mitigation Protocols

Redis Out-of-Memory: Too many active matching tickets. Mitigation: Set expiration TTL on ticket keys.

6.5.6 Estimated Implementation Complexity

High (4/5).

6.6 Phase 5: Matches & Progression (Lifecycle, Scores, ELO deltas)

6.6.1 Objectives

Codify match execution states, score validation logic, and transactional ELO ledger additions.

6.6.2 Tangible Architectural Deliverables

State machine code configuration, transaction logic adding entries to `EloLedger` securely.

6.6.3 Strict Technical Dependencies

Phase 4 real-time gates.

6.6.4 Measurable Acceptance Criteria

Submitting results transitions match states, alters player ratings, and writes logging data trace vectors.

6.6.5 Identified Risks & Mitigation Protocols

Rating Inflation: Exploit vectors in ELO delta calculations. Mitigation: Cap maximum rating changes per match to ± 32 ELO.

6.6.6 Estimated Implementation Complexity

Medium (3/5).

6.7 Phase 6: Events & Tournaments (Brackets, Standings)

6.7.1 Objectives

Deploy competition configurations, tournament bracket generation engine, and automated score aggregations.

6.7.2 Tangible Architectural Deliverables

Bracket calculation engine code structures, tournament standings calculator modules.

6.7.3 Strict Technical Dependencies

Phase 5 match progressions.

6.7.4 Measurable Acceptance Criteria

Bracket updates recalculate layout configurations, winners advance, and tournament metrics log data values.

6.7.5 Identified Risks & Mitigation Protocols

Deadlocks on Bracket recalculations: Mitigation: Recalculate standings asynchronously in BullMQ queues.

6.7.6 Estimated Implementation Complexity

High (4/5).

6.8 Phase 7: Social & Moderation (Chat, Reports, Sanctions)

6.8.1 Objectives

Add communications channels, match dispute tools, and user status controls.

6.8.2 Tangible Architectural Deliverables

WebSocket event handlers for match chats, dispute creation APIs, and user block constraint filters.

6.8.3 Strict Technical Dependencies

Phase 6 tournament events.

6.8.4 Measurable Acceptance Criteria

Active disputes prevent corresponding ELO progression updates. Sanctioned accounts fail to execute matchmaking queues.

6.8.5 Identified Risks & Mitigation Protocols

Chat Spam: Mitigation: Rate limit WebSocket chat events using Redis token bucket limits.

6.8.6 Estimated Implementation Complexity

Medium (3/5).

6.9 Phase 8: Frontend (PWA strategies, Offline, Desktop Web)

6.9.1 Objectives

Assemble responsive mobile and desktop client interfaces with service workers, local stores, and offline caches.

6.9.2 Tangible Architectural Deliverables

Service worker offline cache control code, Capacitor packaging configs, Zustand local store logic.

6.9.3 Strict Technical Dependencies

Phase 7 social gateways.

6.9.4 Measurable Acceptance Criteria

Mobile application launches successfully without active network connection. Push events propagate.

6.9.5 Identified Risks & Mitigation Protocols

Cache Invalidation: Clients run old code versions. Mitigation: Service worker forces reload on update detection.

6.9.6 Estimated Implementation Complexity

High (4/5).

6.10 Phase 9: Production Readiness (Security, Load Testing, Backups)

6.10.1 Objectives

Harden backend APIs, perform load tests on websocket structures, and verify failover steps.

6.10.2 Tangible Architectural Deliverables

API gateway configurations, k6 scaling run scripts, PostgreSQL disaster recovery plans.

6.10.3 Strict Technical Dependencies

Phase 8 application builds.

6.10.4 Measurable Acceptance Criteria

Application backend stays responsive under simulated load. Backup restoration verifies clean targets.

6.10.5 Identified Risks & Mitigation Protocols

DB CPU Spikes: Slow queries during peak load. Mitigation: Implement PgBouncer and optimize database index targets.

6.10.6 Estimated Implementation Complexity

Medium (3/5).

6.11 Phase 10: Production Launch (Rollout, Incident procedures)

6.11.1 Objectives

Deploy production infrastructure, transfer data inputs, and activate monitoring loops.

6.11.2 Tangible Architectural Deliverables

Production launch pipelines, incident response playbooks, metrics tracking dashboards.

6.11.3 Strict Technical Dependencies

Phase 9 testing success.

6.11.4 Measurable Acceptance Criteria

System handles 100% live production traffic. Observability indicators signal system status parameters.

6.11.5 Identified Risks & Mitigation Protocols

Critical Failures at Launch: Mitigation: Automated rollbacks via AWS traffic weights shifts (Canary deployments).

6.11.6 Estimated Implementation Complexity

Medium (3/5).

Chapter 7

Comprehensive Verification & Testing Strategy

To ensure long-term stability, GameHub uses a comprehensive testing pipeline mapped directly to each development phase.

7.1 Unit Testing (Phases 2-7)

Unit tests isolate domain business rules from databases and networking frameworks.

- **Framework**: Jest.
- **Scope**: Pure domain models, value objects, ELO rating math algorithms, bracket composition generators.
- **Target Coverage**: 90% statement coverage across domain services.

```
describe('EloRatingService', () => {
  it('should calculate correct rating changes for balanced matches', () => {
    const playerA = { rating: 1500 };
    const playerB = { rating: 1500 };
    const outcome = EloService.calculate(playerA, playerB, { winner: 'A' });
    expect(outcome.playerADelta).toBe(16);
    expect(outcome.playerBDelta).toBe(-16);
  });
});
```

Listing 7.1: ELO Calculation Unit Test

7.2 Integration Testing (Phases 2-7)

Integration tests evaluate interactions with databases, cache engines, and task handlers.

- **Framework**: Testcontainers with PostgreSQL and Redis.
- **Scope**: TypeORM repository layers, transactional lock execution, Redis lock acquisitions, BullMQ job triggers.

```
describe('PlayAreaReservationRepository', () => {
  it('should prevent double bookings in transaction block', async () => {
    const resA = repository.create({ areaId: 1, timeslot: '09:00-10:00' });
    const resB = repository.create({ areaId: 1, timeslot: '09:00-10:00' });
    await expect(Promise.all([resA, resB])).rejects.toThrow();
```

```
  });
});
```

Listing 7.2: PlayAreaReservation Integration Test

7.3 API Testing (Phases 2-7)

API testing validates routing layers, request schemas, authentication permissions, and HTTP status codes.

- **Framework**: Supertest with NestJS application instance.
- **Scope**: Controller responses, validation filters, access guards, header validation.

7.4 Real-Time WebSocket Testing (Phase 4)

Real-time gateway testing verifies event delivery patterns and connection logic under pressure.

- **Framework**: `socket.io-client` wrapper.
- **Scope**: Room groupings, event delivery verification, heartbeat mechanics.

7.5 Security Testing (Phase 9)

Security tests check for common vulnerabilities and access issues.

- **Tooling**: OWASP ZAP, snyk audits.
- **Scope**: JWT signature tampering checks, SQL injections, XSS checks, dependency audits, RBAC bypasses.

7.6 Performance & Load Testing (Phase 9)

Load tests verify platform behaviors under expected traffic peaks.

- **Framework**: k6.
- **Scope**: Matchmaking queue operations, websocket connection limits, Postgres connection saturation points.

7.7 End-to-End (E2E) Testing (Phase 8)

E2E tests simulate user flows across both PWA mobile containers and desktop screens.

- **Framework**: Playwright.
- **Scope**: Registration, match flows, offline caching simulation, service worker updates.

Chapter 8

Production Readiness & Day-2 Operations

This chapter details monitoring setups, operational reliability policies, backup structures, security configs, and optimization parameters.

8.1 System Metrics Monitoring & Alerting

Observability relies on OpenTelemetry collectors, Prometheus scraping engines, and Grafana dashboards.

8.1.1 System Metrics to Track

Metric Category	Target Metric Name	Warning Threshold
Database Connection Pool	`db_pool_active_connections`	$>$ 80% pool capacity
Redis Cache Latency	`redis_command_latency_seconds`	$>$ 5ms
BullMQ Process Duration	`bullmq_job_processing_seconds`	$>$ 30s
Node Event Loop Lag	`nodejs_eventloop_lag_seconds`	$>$ 50ms
WebSocket Connections	`websocket_active_connections`	$>$ 15,000 clients

8.2 Structured Logging Standard

All system components output logs to `stdout` as structured JSON events. Log processors (Vector/FluentBit) push these events to Loki databases.

```
{
  "timestamp": "2026-06-20T23:30:00.123Z",
  "level": "error",
  "context": "EloProgressionService",
  "message": "ELO transaction rollback",
  "traceId": "t_83210984aef09b",
  "error": {
    "message": "Unique constraint violation on EloLedger",
    "stack": "Error: Unique constraint..."
  }
}
```

Listing 8.1: Structured Logging JSON Event

8.3 Disaster Recovery & Backups

To protect system state, PostgreSQL and Redis employ automated recovery setups:

- **PostgreSQL Backups**: Daily full snapshots stored in AWS S3 across multiple zones. WAL archiving configured to allow Point-in-Time Recovery (PITR) to any second within the past 14 days.
- **Recovery Metrics**:
  - RTO (Recovery Time Objective): $<$ 1 hour.
  - RPO (Recovery Point Objective): $<$ 1 minute (enabled by WAL archives).
- **Redis Recovery**: RDB snapshots configured for disaster states, combined with active replica nodes in secondary zones.

8.4 Operational Security Checklist

Prior to deployment, the system must conform to the following:

1. **CORS Configuration**: Express endpoints restrict origins to authorized app domains.
2. **HTTP Headers**: HelmetJS enabled to inject secure configurations (X-Frame-Options, CSP, HSTS).
3. **TLS Layering**: TLS 1.3 enforced for all edge connections. SSL termination configured on Cloudflare.
4. **Rate Limiting**: Inbound rate limiters reject IPs exceeding 100 requests per minute on public API endpoints.

8.5 Cost Optimization Strategies

- **Connection Pooling**: PgBouncer handles database connection overhead, capping backend connection footprint.
- **Cache Policies**: Volatile data keys in Redis use strict TTL limits to prevent memory footprint bloat.
- **Asset Delivery**: Cloudflare caches static assets, reducing network compute overhead.

8.6 OpenTelemetry Observability Configuration

Observability relies on OpenTelemetry collectors. The application uses a unified SDK bootstrap module in TypeScript for distributed tracing and performance metrics collection:

```
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-
    instrumentations-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-grpc';
import { OTLPMetricExporter } from '@opentelemetry/exporter-metrics-otlp-grpc';
import { PeriodicExportingMetricReader } from '@opentelemetry/sdk-metrics';

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT || 'http://localhost:4317',
  }),
```

```
  metricReader: new PeriodicExportingMetricReader({
    exporter: new OTLPMetricExporter({
      url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT || 'http://localhost:4317',
    }),
    exportIntervalMillis: 60000,
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();

process.on('SIGTERM', () => {
  sdk.shutdown()
    .then(() => console.log('OTel SDK shut down successfully'))
    .catch((error) => console.log('Error shutting down OTel SDK', error))
    .finally(() => process.exit(0));
});
```

Listing 8.2: OpenTelemetry SDK Bootstrap Configuration

8.7 Cluster Scaling Bounds & Policies

To ensure high availability and cost efficiency under varying student traffic, the NestJS containerized application runs on AWS ECS Fargate with the following auto-scaling bounds:

- **Minimum Tasks**: 2 containers (provisioned across separate Availability Zones for high availability).
- **Maximum Tasks**: 10 containers.
- **Scale-Out Policy**: Triggers when average CPU utilization exceeds 70% or memory utilization exceeds 80% sustained over a 3-minute evaluation window. Adds 2 tasks.
- **Scale-In Policy**: Triggers when average CPU utilization drops below 30% and memory utilization drops below 40% sustained over a 15-minute evaluation window. Removes 1 task.
- **Database Connection Limits**: PgBouncer connection pooling caps maximum active backend connections to 500, aligning database resources with the peak scaling limit (10 tasks $\times$ 50 connections/task).

Chapter 9

Architectural Visualizations & Diagrams

This section contains visual representations mapping system flows, structures, and dependencies, combining baseline V1 specifications directly alongside production-grade V2 configurations.

9.1 Hexagonal Dependency Flow

Figure 9.1 displays the flow of dependencies from the external transport layer to the core domain.

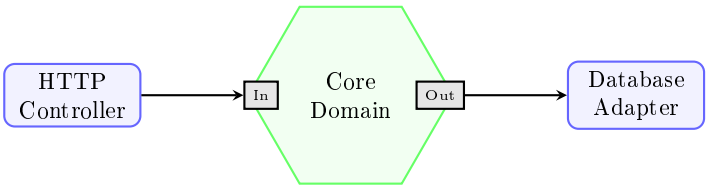

Figure 9.1: Hexagonal Architecture Dependency Flow

9.2 Domain Interaction Map

Figure 9.2 outlines how the core business domain layers communicate.

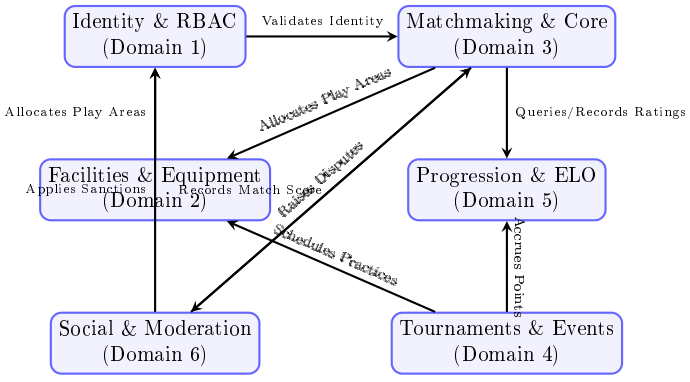

Figure 9.2: Domain Interaction Map

9.3 Entity Relationship Diagram (ERD)

Figure 9.3 represents the primary entity relationship layout of the system.

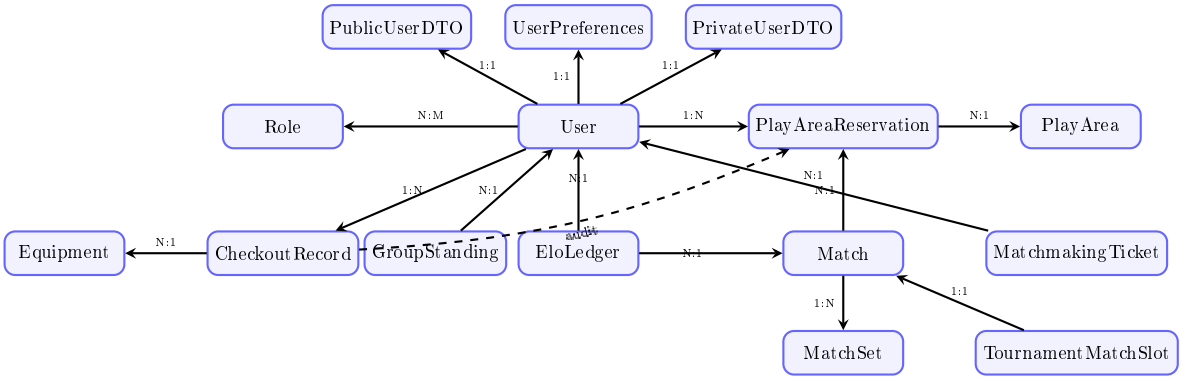

Figure 9.3: Full Entity Relationship Diagram (ERD) Layout

9.4 Matchmaking Sequence

Figure 9.4 displays the message flow during a successful player matchmaking request.

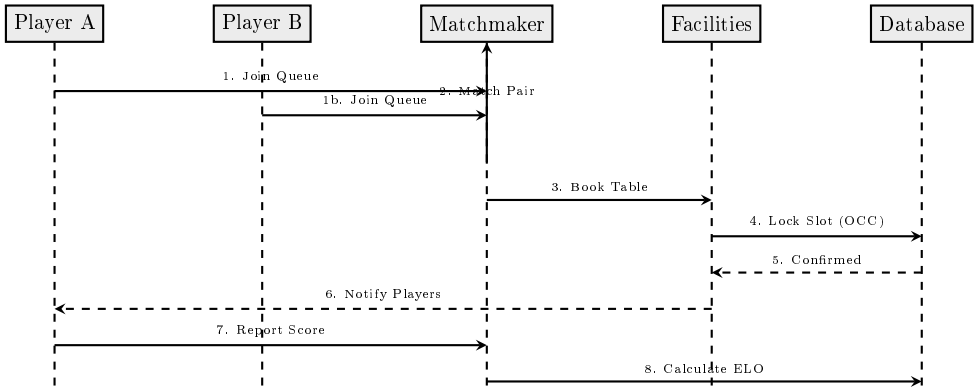

Figure 9.4: Matchmaking Flow and Facility Reservation Sequence

9.5 Consolidated Backend Topology

Figure 9.5 outlines the physical instances and load balancing structure of scaled nodes.

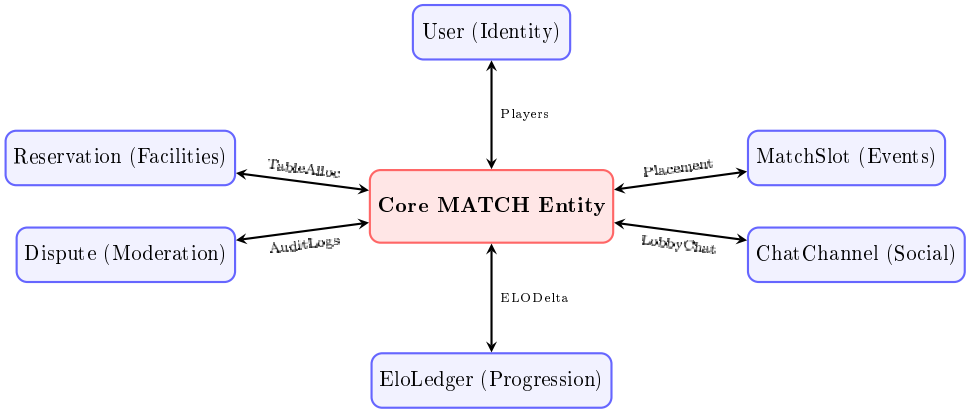

Figure 9.5: Consolidated Match-Centric Domain Linkage

9.6 Client Connectivity State Machine

Figure 9.6 defines the client connection lifecycles, retries, and jitter policies, which are cross-referenced with the Zustand store and Capacitor mobile hooks.

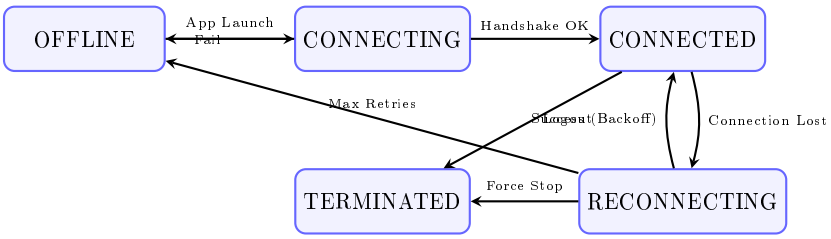

Figure 9.6: Client Connectivity State Machine

9.7 Concurrency Lock Handling Sequence

Figure 9.7 showcases optimistic concurrency control handling when reserving resources with capacity bounds. Under this flow, Thread A commits successfully, updating version from 0 to 1, while Thread B aborts with an `OptimisticLockException` and immediately triggers the matchmaking priority re-queue.

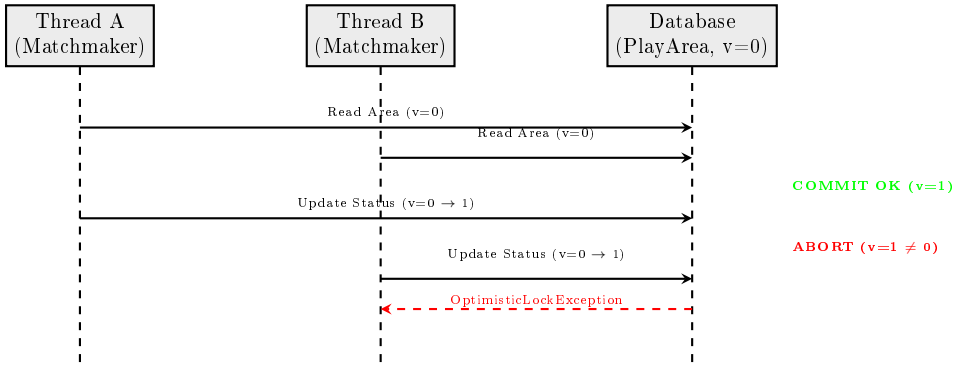

Figure 9.7: Optimistic Lock Conflict handling for tableCount = 1